



Security Audit Report for DGRID

Date: February 10, 2026 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction	1
1.1 About the Audit Target	1
1.2 Disclaimer	2
1.3 Procedure of Auditing	2
1.3.1 Security Issues	2
1.3.2 Additional Recommendation	3
1.4 Security Model	3
Chapter 2 Findings	5
2.1 Security Issue	5
2.1.1 Lack of pool update in function <code>setRewardPerBlockByIndex()</code>	5
2.1.2 Lack of start block tracking for new <code>fixedRewardToken</code>	6
2.1.3 Fixed reward accrual during pause	7
2.1.4 Lack of <code>nonce</code> in server signature scheme	8
2.2 Recommendation	9
2.2.1 Implement removal mechanism for supported tokens	9
2.2.2 Add an empty check for <code>_nodeIds</code> in function <code>unstake()</code>	10
2.3 Note	11
2.3.1 Potential centralization risks	11
2.3.2 Trust assumptions on <code>TDGAI</code> transfer calldata	11

Report Manifest

Item	Description
Client	DgridAI
Target	DGRID

Version History

Version	Date	Description
1.0	February 10, 2026	First release

Signature



Digitally signed by: BlockSec (<https://blocksec.com>)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2026-02-10 10:46:35+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at [Email](#), [Twitter](#) and [Medium](#).

Chapter 1 Introduction

1.1 About the Audit Target

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The audit target (hereinafter referred to as the Target) of this audit is the code repository ¹ of DGRID of DgridAI.

The DGrid contracts implement a comprehensive on-chain billing and incentive system for the DGrid network, covering node staking, reward distribution, payment processing, and competition-based incentives. Users purchase DgridNode and stake them via the Dgrid-StakePool contract, which manages node staking and distributes rewards under both fixed and variable models, while enforcing penalties on misbehaving or abnormal nodes through a node jailing mechanism to ensure economic security and system stability. Within this system, the DgridTopUp contract allows users to top up using native tokens, tDGAi, or supported stablecoins, forwarding funds to the designated development address while recording corresponding deposits on-chain. In addition, the DgridAIArena contract handles AI Arena reward settlement, where winners determined by off-chain competition or task results are registered by the server and subsequently claim their rewards on-chain.

Note this audit only focuses on the smart contracts in the following directories/files:

- contracts/DGAi.sol
- contracts/DgridAIArena.sol
- contracts/DgridNode.sol
- contracts/DgridStackPool
- contracts/DgridTopUp.sol

Other files are not within the scope of the audit. Additionally, all dependencies of the Target are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version ([Version 1](#)), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version ([Version 0](#)), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
DGRID	Version 1	8f080ce40622cb37b4206af1b6cc920ffb77ed63
	Version 2	e54c6480b9ee70d03b03c517769d397f9e887da0

¹<https://github.com/dgridai/DGRID>

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the Target, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of the Target.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan the Target with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of the Target and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

- * Access control
- * Permission management
- * Whitelist and blacklist mechanisms
- * Initialization consistency
- * Improper use of the proxy system
- * Reentrancy
- * Denial of Service (DoS)
- * Untrusted external call and control flow
- * Exception handling
- * Data handling and flow
- * Events operation
- * Error-prone randomness

- * Oracle security
- * Business logic correctness
- * Semantic and functional consistency
- * Emergency mechanism
- * Economic and incentive impact

1.3.2 Additional Recommendation

- * Gas optimization
- * Code quality and style



Note The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology ² and Common Weakness Enumeration ³. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	High	High	Medium
	Low	Medium	Low
		High	Low
		Likelihood	

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.

²https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

³<https://cwe.mitre.org/>

- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

Chapter 2 Findings

In total, we found **four** potential security issues. Besides, we have **two** recommendations and **two** notes.

- High Risk: 2
- Low Risk: 2
- Recommendation: 2
- Note: 2

ID	Severity	Description	Category	Status
1	High	Lack of pool update in function <code>setRewardPerBlockByIndex()</code>	Security Issue	Fixed
2	High	Lack of start block tracking for new <code>fixedRewardToken</code>	Security Issue	Fixed
3	Low	Fixed reward accrual during pause	Security Issue	Fixed
4	Low	Lack of <code>nonce</code> in server signature scheme	Security Issue	Fixed
5	-	Implement removal mechanism for supported tokens	Recommendation	Fixed
6	-	Add an empty check for <code>_nodeIds</code> in function <code>unstake()</code>	Recommendation	Fixed
7	-	Potential centralization risks	Note	-
8	-	Trust assumptions on <code>TDGAI</code> transfer call-data	Note	-

The details are provided in the following sections.

2.1 Security Issue

2.1.1 Lack of pool update in function `setRewardPerBlockByIndex()`

Severity High

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the `DgridStakePool` contract, the function `setRewardPerBlockByIndex()` allows the owner to update the `rewardPerBlock` value for a specific reward token. However, the function does not invoke function `updatePool()` to settle accrued rewards before applying the new reward rate.

As a result, rewards accumulated between the last pool update and the execution of function `setRewardPerBlockByIndex()` are not finalized using the previous `rewardPerBlock`. When the pool is later updated, these already accrued rewards are instead settled using the newly updated rate, effectively replacing the reward rate that should have applied during the earlier staking period.

```
513 //admin: adjust reward per block
514 function setRewardPerBlockByIndex(
515     uint256 _rewardTokenIndex,
```

```
516     uint256 _rewardPerBlock
517 ) external onlyOwner {
518     require(
519         _rewardTokenIndex < rewardTokenInfos.length,
520         "index out of range"
521     );
522     require(_rewardPerBlock > 0, "reward per block is zero");
523     emit UpdateRewardPerBlock(
524         address(rewardTokenInfos[_rewardTokenIndex].rewardToken),
525         rewardTokenInfos[_rewardTokenIndex].rewardPerBlock,
526         _rewardPerBlock
527     );
528     rewardTokenInfos[_rewardTokenIndex].rewardPerBlock = _rewardPerBlock;
529 }
```

Listing 2.1: contracts/DgridStakePool.sol

Impact This flaw may cause pending staking rewards to be computed using an updated `rewardPerBlock`, leading to inconsistency and unfair reward distribution.

Suggestion Invoke the function `updatePool()` at the beginning of the function `setRewardPerBlockByIndex()` to settle all accrued rewards using the previous `rewardPerBlock` before applying the updated rate.

2.1.2 Lack of start block tracking for new `fixedRewardToken`

Severity High

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the `DgridStakePool` contract, the function `addFixedRewardToken()` allows the owner to introduce new fixed reward tokens for `DgridNode` stakers. Fixed reward accrual is based on the per-user variable `fixedLatestRewardBlock`, which is updated to the current block whenever a user performs state-changing actions such as deposit, unstake, or harvest via the internal function `_resetUserFixedLatestRewardBlock()`.

When a new fixed reward token is added after users have already staked, reward calculation for the new token follows the same mechanism and starts from each user's `fixedLatestRewardBlock`, i.e., the block at which the user last updated their staking state, rather than from the block at which the new fixed reward token was added.

As a result, rewards for the newly added fixed reward token may be calculated over a period during which the token was not yet active.

```
690 function addFixedRewardToken(
691     address _rewardToken,
692     uint256 _rewardPerNodePerBlock
693 ) external onlyOwner {
694     require(_rewardToken != address(0), "reward token is zero address");
695     require(
696         _rewardPerNodePerBlock > 0,
697         "reward per node per block is zero"
698     );
699 }
```

```
698     );
699     fixedRewardTokenInfos.push(
700         FixedRewardTokenInfo({
701             rewardToken: ERC20(_rewardToken),
702             rewardPerNodePerBlock: _rewardPerNodePerBlock,
703             finalRewardBlock: 0
704         })
705     );
706 }
```

Listing 2.2: contracts/DgridStakePool.sol

Impact Users who staked prior to the addition of a new fixed reward token may receive rewards for blocks before the token's introduction, leading to excess reward distribution and unnecessary additional payouts by the project.

Suggestion Revise the corresponding logic.

2.1.3 Fixed reward accrual during pause

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the `DgridStakePool` contract, the function `pendingFixedReward()` calculates each user's fixed rewards based on the per-user variable `fixedLatestRewardBlock` and the elapsed reward blocks determined by the function `_calculateFixedRewardBlocks()`. However, the calculation does not account for the pool's paused state. If the contract is paused for a period of time, `fixedLatestRewardBlock` is not adjusted to exclude the paused interval, and function `pendingFixedReward()` continues to count blocks across the pause window.

As a result, users' fixed rewards may include rewards accrued during the paused period, which may be inconsistent with the expected semantics of pause (i.e., temporarily halting reward accrual).

```
709     function pendingFixedReward(
710         address _user
711     )
712     public
713     view
714     returns (
715         address[] memory rewardTokens,
716         uint256[] memory rewards,
717         uint256[] memory paidRewards
718     )
719     {
720         uint256 n = fixedRewardTokenInfos.length;
721         rewardTokens = new address[](n);
722         rewards = new uint256[](n);
723         paidRewards = new uint256[](n);
724
725         if (n == 0) {
```

```
726     return (rewardTokens, rewards, paidRewards);
727 }
728
729 if (block.number < startBlock) {
730     for (uint256 i = 0; i < n; i++) {
731         rewards[i] = 0;
732         rewardTokens[i] = address(fixedRewardTokenInfos[i].rewardToken);
733         paidRewards[i] = 0;
734     }
735     return (rewardTokens, rewards, paidRewards);
736 }
737
738 UserInfo storage user = userInfo[_user];
739
740 uint256 latestRewardBlock = fixedLatestRewardBlock[_user];
741 if (latestRewardBlock == 0) {
742     latestRewardBlock = startBlock;
743 }
744 for (uint256 i = 0; i < n; i++) {
745     uint256 finalBlock = fixedRewardTokenInfos[i].finalRewardBlock;
746     uint256 rewardBlocks = _calculateFixedRewardBlocks(
747         latestRewardBlock,
748         finalBlock
749     );
750     uint256 unpaid = i < fixedUnpaidRewards[_user].length
751         ? fixedUnpaidRewards[_user][i]
752         : 0;
753
754     rewards[i] =
755         unpaid +
756         rewardBlocks *
757         fixedRewardTokenInfos[i].rewardPerNodePerBlock *
758         user.amount;
759     rewardTokens[i] = address(fixedRewardTokenInfos[i].rewardToken);
760     paidRewards[i] = i < fixedPaidRewards[_user].length
761         ? fixedPaidRewards[_user][i]
762         : 0;
763 }
764 return (rewardTokens, rewards, paidRewards);
765 }
```

Listing 2.3: contracts/DgridStakePool.sol

Impact Users can receive fixed rewards for blocks within the paused period.

Suggestion Revise the corresponding logic.

2.1.4 Lack of nonce in server signature scheme

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the `DgridStakePool` contract, the functions `deposit()` and `unjailNodes()` rely on off-chain server signatures to authorize user actions. These signatures are validated on-chain by checking that they are signed by the authorized server address and that the provided `_expireTime` has not yet passed.

However, the signed payload does not include a unique `nonce`. Instead, the signature is derived solely from static parameters, including `block.chainid`, the input node identifiers, the staker or owner address, the expiration time, and a fixed action identifier. As a result, a valid signature can be reused multiple times as long as it remains within its validity window.

This design allows signatures to be replayed, enabling the same authorization to be executed more than once without requiring a fresh server approval.

```
329     // server sign check
330     bytes32 ethSignedMessageHash = MessageHashUtils.toEthSignedMessageHash(
331         abi.encode(
332             block.chainid,
333             _nodes,
334             _staker,
335             _expireTime,
336             ACTION_DEPOSIT
337         )
338     );
```

Listing 2.4: contracts/DgridStakePool.sol

```
431     bytes32 ethSignedMessageHash = MessageHashUtils.toEthSignedMessageHash(
432         abi.encode(
433             block.chainid,
434             _nodeIds,
435             _owner,
436             _expireTime,
437             ACTION_UNJAIL
438         )
439     );
```

Listing 2.5: contracts/DgridStakePool.sol

Impact If a node is unjailed using a valid signature and is subsequently re-jailed within the same expiration window, the node owner could reuse the earlier signature to unjail the node again without obtaining a new authorization from the server.

Suggestion Include a unique `nonce` in the signed payload and track its usage on-chain to ensure that each server signature can only be used once.

2.2 Recommendation

2.2.1 Implement removal mechanism for supported tokens

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the `DgridTopUp` contract, the owner can add supported stablecoins for payment via the function `addSupportToken()`. However, the contract does not provide a corresponding function to remove a previously added supported token.

As a result, if a supported stablecoin becomes deprecated, compromised, loses its peg, or is otherwise no longer intended to be accepted, it cannot be revoked from the supported token set. This limitation reduces operational flexibility and may lead to unintended payment acceptance or increased operational risk over time.

Suggestion Provide a function that allows the owner to remove a supported token from the accepted token set, enabling the contract to respond to changes in token status or operational requirements.

2.2.2 Add an empty check for `_nodeIds` in function `unstake()`

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the contract `DgridStakePool`, the function `unstake()` allows users to withdraw staked nodes by supplying the input `_nodeIds`. However, the input `_nodeIds` is not validated to ensure it is non-empty. If `_nodeIds` is an empty array, the function still executes state-changing logic such as reward accrual and debt resetting, consumes unnecessary gas, and emits the event `Unstake()` with a zero amount, despite no nodes being unstaked.

```
357 function unstake(  
358     uint256[] memory _nodeIds  
359 ) external nonReentrant whenUnstakeEnabled whenNotPaused {  
360     require(block.number >= startBlock, "not started");  
361     for (uint256 i = 0; i < _nodeIds.length; i++) {  
362         require(dgridNode.ownerOf(_nodeIds[i]) == msg.sender, "not owner");  
363         require(!dgridNode.isJailed(_nodeIds[i]), "node is already jailed");  
364         require(dgridNode.isStaked(_nodeIds[i]), "node is not staked");  
365     }  
366     updatePool();  
367     _ensureUserInfoLen(msg.sender);  
368     _ensureUserFixedRewardsInfoLen(msg.sender);  
369     _accrueUnpaid(msg.sender);  
370     _accrueFixedUnpaid(msg.sender);  
371     UserInfo storage user = userInfo[msg.sender];  
372     require(user.amount >= _nodeIds.length, "not enough staked");  
373     user.amount -= _nodeIds.length;  
374     totalStaked -= _nodeIds.length;  
375     _resetDebt(msg.sender);  
376     _resetUserFixedLatestRewardBlock(msg.sender);  
377     dgridNode.unstake(_nodeIds);  
378     emit Unstake(msg.sender, _nodeIds.length);  
379 }
```

Listing 2.6: contracts/DgridStakePool.sol

Suggestion Add an empty check for the `_nodeIds` in function `unstake()`.

2.3 Note

2.3.1 Potential centralization risks

Introduced by [Version 1](#)

Description In this project, several privileged roles (e.g., [Owner](#) and [Server](#)) can conduct sensitive operations, which introduces potential centralization risks. For example, the [Server](#) can invoke the privileged function `jailNodes()` to lock user-staked nodes, preventing them from being unstaked or transferred. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

2.3.2 Trust assumptions on TDGAI transfer calldata

Introduced by [Version 1](#)

Description When transferring accrued [TDGAI](#) rewards via function `transferTdgai()`, the sender may supply arbitrary calldata if the receiver is a contract that declares support for [ITdgaiTransferReceiver](#) via [ERC-165](#). This calldata is forwarded directly to the receiver's `onTdgaiTransfer()` callback function and is not validated by the protocol. As a result, receiver contracts must not assume the correctness, authenticity, or safety of the provided calldata and should perform comprehensive validation before relying on it in any business logic. Any security assumptions based on untrusted calldata may lead to unintended behavior or vulnerabilities.

